

Patrones Creacionales y Seguridad

Proyecto: Gestión de Eventos y Donaciones - Fundación Cudeca

Equipo Turing

1. Introducción

El objetivo de este desarrollo ha sido desacoplar la lógica de negocio mediante patrones de diseño, así como asegurar la integridad de los datos y la funcionalidad de los periféricos mediante la implementación de protocolos de seguridad robustos.

2. Patrones Creacionales Aplicados

A continuación se detallan los patrones implementados para la estructuración del código.

A. Singleton

Clase / Herramienta: AppConfig (en patterns.py)

- Justificación:

Necesidad de gestionar la configuración de rutas de subida de archivos de manera centralizada.

Problema: Inicialización múltiple de directorios y rutas dispersas ("hardcoded") en el código.

Solución: Garantiza una única instancia de configuración global que verifica los directorios una sola vez al inicio.

B. Factory Method

Clase / Herramienta: PaymentFactory (Creador) y PaymentProcessor (Interfaz)

- Justificación:

El sistema requiere procesar pagos de diferentes tipos (Tarjeta, Bizum, PayPal) de forma escalable.

Problema: El uso de condicionales (if/else) en los controladores viola el principio Open/Closed y dificulta el mantenimiento.

Solución: Encapsula la creación del procesador de pagos. El cliente solicita un método sin conocer la lógica interna de la pasarela.

C. Builder

Clase / Herramienta: EventoBuilder (en patterns.py)

- Justificación:

La creación de un Evento es compleja (datos, fechas, precios, ficheros de imagen).

Problema: Controladores sobrecargados con lógica de validación, conversión de tipos y gestión de sistema de archivos.

Solución: Construye el objeto paso a paso, limpiando el controlador y encapsulando la gestión de archivos.

D. Abstract Factory

Clase / Herramienta: TransactionFactory, EventTransactionFactory, RaffleTransactionFactory

- Justificación:

Necesidad de tratar compras de entradas y rifas de forma polimórfica, aunque generen productos distintos.

Problema: Los Eventos generan Entradas + QRs. Las Rifas generan Boletos sin QR.

Solución: Garantiza la coherencia entre los objetos creados (Registro BD + Token de Acceso) según la familia de la transacción.

E. Prototype

Clase / Herramienta: Evento, Rifa (método clone() en models.py)

- Justificación:

Necesidad administrativa de duplicar eventos recurrentes.

Problema: La reintroducción manual de datos es ineficiente y propensa a errores humanos.

Solución: Permite clonar una instancia existente en memoria para guardarla como una nueva entidad.

3. Seguridad e Infraestructura

Medidas implementadas para proteger los datos y habilitar funcionalidades hardware.

A. Encriptación de Credenciales

Clase / Herramienta: Werkzeug Security (generate_password_hash, check_password_hash)

- Justificación:

Protección crítica de la identidad de los usuarios y administradores en la base de datos.

Problema: Almacenar contraseñas en texto plano es una vulnerabilidad crítica; en caso de filtración de la base de datos (SQL Injection), las cuentas quedarían expuestas.

Solución: Uso del algoritmo de hashing pbkdf2:sha256 con sal (salt). La contraseña real nunca se guarda; solo se almacena su huella criptográfica irrecuperable.

B. Protocolo Seguro HTTPS

Clase / Herramienta: Flask SSL Context (Ad-hoc) / Librería Cryptography

- Justificación:

Requisito técnico indispensable para el funcionamiento del escáner QR y la integridad de los datos.

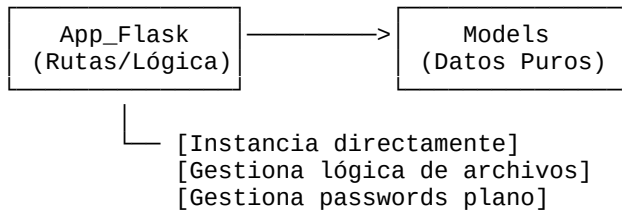
Problema: Los navegadores modernos bloquean el acceso a la API de la cámara (getUserMedia) en conexiones no seguras (HTTP). Además, los datos viajarían sin cifrar.

Solución: Implementación de certificados SSL autofirmados (adhoc) para el entorno de desarrollo, habilitando el cifrado TLS y el permiso de uso de la cámara.

4. Diagramas de Clases (Refactorización)

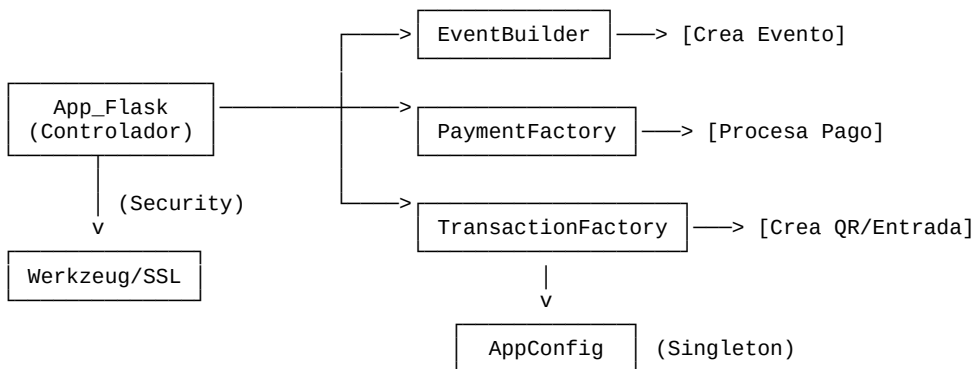
Antes de la Refactorización (Diseño Monolítico)

En este escenario, app.py asumía toda la responsabilidad, generando alto acoplamiento.



Después de la Refactorización (Patrones + Seguridad)

Arquitectura distribuida con delegación de responsabilidades y capa de seguridad.



5. Conclusión

La aplicación de patrones de diseño ha transformado el sistema en una arquitectura robusta y escalable. Adicionalmente, la integración de hashing de contraseñas y protocolo HTTPS asegura que la plataforma cumple con los estándares modernos de privacidad y funcionalidad técnica.